

Clean Code is code that is easy to **read, understand, maintain, test, and extend** by other developers (or even yourself after 6 months).

The term became popular through the book Clean Code by Robert C. Martin.

Simple Definition

Clean Code is code that clearly expresses its intent and is easy to change without introducing bugs.

When another developer opens your code, they should understand it quickly without needing a long explanation.

Example 1: Bad Code

```
public BigDecimal c(List<Order> o) {  
    BigDecimal t = BigDecimal.ZERO;  
  
    for (Order x : o) {  
        t = t.add(x.getP());  
    }  
  
    return t;  
}
```

Questions:

- What is c?
- What is o?
- What is x?
- What is P?

Nobody knows.

Example 2: Clean Code

```
public BigDecimal calculateTotalAmount(List<Order> orders) {  
    BigDecimal totalAmount = BigDecimal.ZERO;  
  
    for (Order order : orders) {  
        totalAmount = totalAmount.add(order.getPrice());  
    }  
  
    return totalAmount;  
}
```

Now the intent is obvious.

Characteristics of Clean Code

1. Meaningful Names

Bad:

```
int d;
```

Good:

```
int daysSinceRegistration;
```

2. Small Methods

Bad:

```
public void processOrder() {  
    // 300 lines  
}
```

Good:

```
public void processOrder() {  
    validateOrder();  
    reserveInventory();  
    createPayment();  
    sendNotification();  
}
```

Each method has one responsibility.

3. Single Responsibility Principle (SRP)

Bad:

```
public class UserService {  
  
    public void register(User user) {  
        validate(user);  
        saveToDatabase(user);  
        sendEmail(user);  
        generateReport(user);  
    }  
}
```

This class does too many things.

Better:

```
public class UserService {  
    private final UserValidator validator;  
    private final UserRepository repository;  
    private final EmailService emailService;  
  
    public void register(User user) {  
        validator.validate(user);  
        repository.save(user);  
    }  
}
```

```
        emailService.sendWelcomeEmail(user);
    }
}
```

4. Avoid Duplicate Code

Bad:

```
if (user.getAge() < 18) {
    throw new IllegalArgumentException();
}
```

```
if (customer.getAge() < 18) {
    throw new IllegalArgumentException();
}
```

Good:

```
private void validateAdult(int age) {
    if (age < 18) {
        throw new IllegalArgumentException();
    }
}
```

5. Proper Class Structure

Bad:

```
UserController
    -> Repository
```

Good:

UserController

-> UserService

-> UserRepository

Clear responsibilities.

6. Avoid Large Classes

Bad:

UserService

Contains:

- Registration
- Login
- OTP
- Password Reset
- Reporting
- Export
- Email

2000+ lines.

Good:

RegisterUserUseCase

LoginUseCase

VerifyOtpUseCase

ResetPasswordUseCase

7. Write Self-Documenting Code

Bad:

```
// Add 7 days to expiration date  
expirationDate = expirationDate.plusDays(7);
```

Comment is unnecessary.

Good:

```
expirationDate = expirationDate.plusDays(OTP_VALIDITY_DAYS);
```

The code explains itself.

Clean Code vs Design Pattern

Many developers confuse these.

Clean Code

Focuses on:

- Readability
- Maintainability
- Simplicity

Example:

```
validateOrder();  
saveOrder();  
sendNotification();
```

Design Pattern

Focuses on:

- Reusable architecture solutions
- Flexible design
- Object collaboration

Examples:

- Factory
- Builder

- Strategy
- Observer
- Facade
- Adapter

A project can use Design Patterns and still be messy if naming and responsibilities are poor.

In Enterprise Projects

When reviewing code, I usually check these questions:

1. Can I understand this method in less than 30 seconds?
2. Does this class have only one responsibility?
3. Are names meaningful?
4. Is there duplicated logic?
5. Can this code be unit tested easily?
6. If requirements change tomorrow, can I modify it safely?

If the answer is mostly **yes**, the code is probably clean.